

MODULE - 1

SUN MON TUE WED THU FRI SAT

2nd

Topic. Date / /

Basic Operational Concepts

- ↳ To perform a given task, an appropriate Pgm consisting of a list of instructions is stored in the memory.
- ↳ Individual instructions are brought from the memory into the processor.
(also data used as operands ^{instr's} are also in the memory.)

Eg: Load R₂, LOC

↓
reads the contents of the a memory location whose address is represented symbolically by the label LOC. and loads them into processor register R₂.

- LOC $\rightarrow$ original contents of LOC are preserved.
- $R_2 \rightarrow$ contents of register R_2 are overwritten.

Execution of an Instruction

- 1) $\hookrightarrow$ Instruction is fetched from the memory into the processor.
- 2) $\hookrightarrow$ Operation to be performed is determined by the control unit.
- 3) $\hookrightarrow$ The operand at LOC is then fetched from the memory into the processor.
- 4) $\hookrightarrow$ Finally the operand is stored in register R_2 .

↳ After operands have been loaded from memory into processor registers arithmetic or logic operations can be performed.

Eg: Add R_4, R_2, R_3

↳ adds the contents of R_2 & R_3 and store it into R_4

↳ R_2 & R_3 's contents are not altered by R_4 is overwritten by sum.

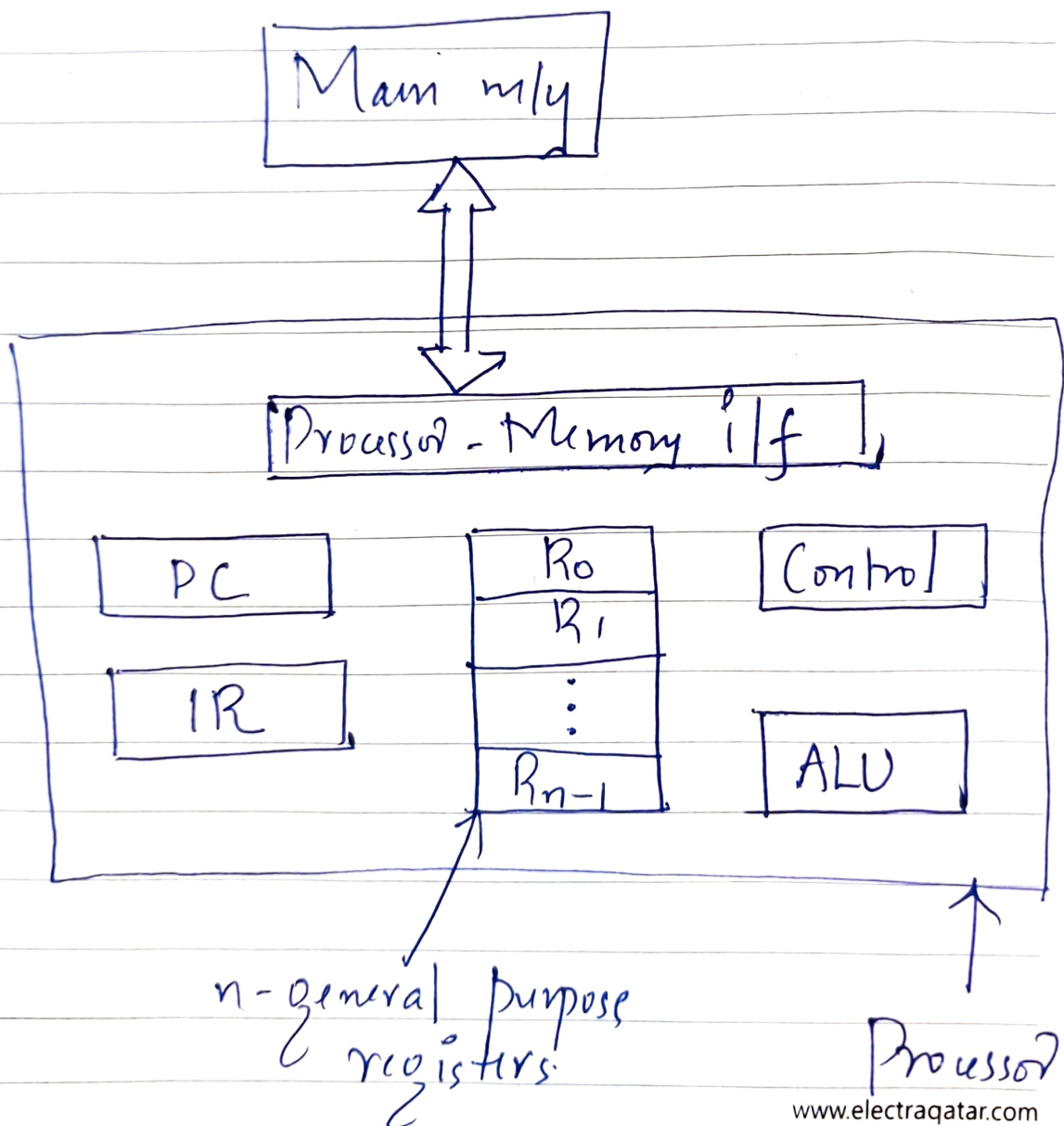
5) After completing the desired operation the results are in processor registers eg: (R_4)

Now it is transferred to the memory using the instructions

Store R_4, LOC
 here, R_4 - preserved
 LOC - overwritten

=> Load & Store instructions
transfers b/w memory & processor

- initiated by sending the address of the desired mem^y location to the mem^y unit.



* Processor contains a number of registers (different purposes).

1) IR → Instruction Register holds the instruction that is currently being executed.

IR - generates the timing signals that control the various processing elements involved in executing the instruction.

(works closely with PC & CU)

- Once fetched from m/b, the instruction is placed into IR.
- This prevents issues where m/b changes before execution finishes.

→ Based on the instruction inside the IR the control unit,

* select ALU operations * enables register writes * manages memory reads/writes

- * Chooses addressing modes
- * Controls branching and jumping.

⇒ The IR is the primary driver of control flow inside the CPU.

2) PC → Program Counter
Specialised Register

- * Contains the mpy address of the next instruction to be fetched and executed

- * PC points to the next instruction that is to be fetched from the mpy. So called as Instruction Pointer

Uses → Ensure CPU, what instr next -
→ Controls pgm flow
→ Ensures correct execution order.

3) General Purpose Registers $\rightarrow R_0$ to R_{n-1}

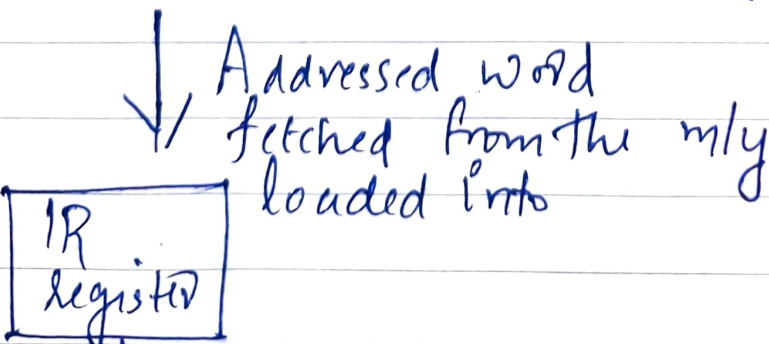
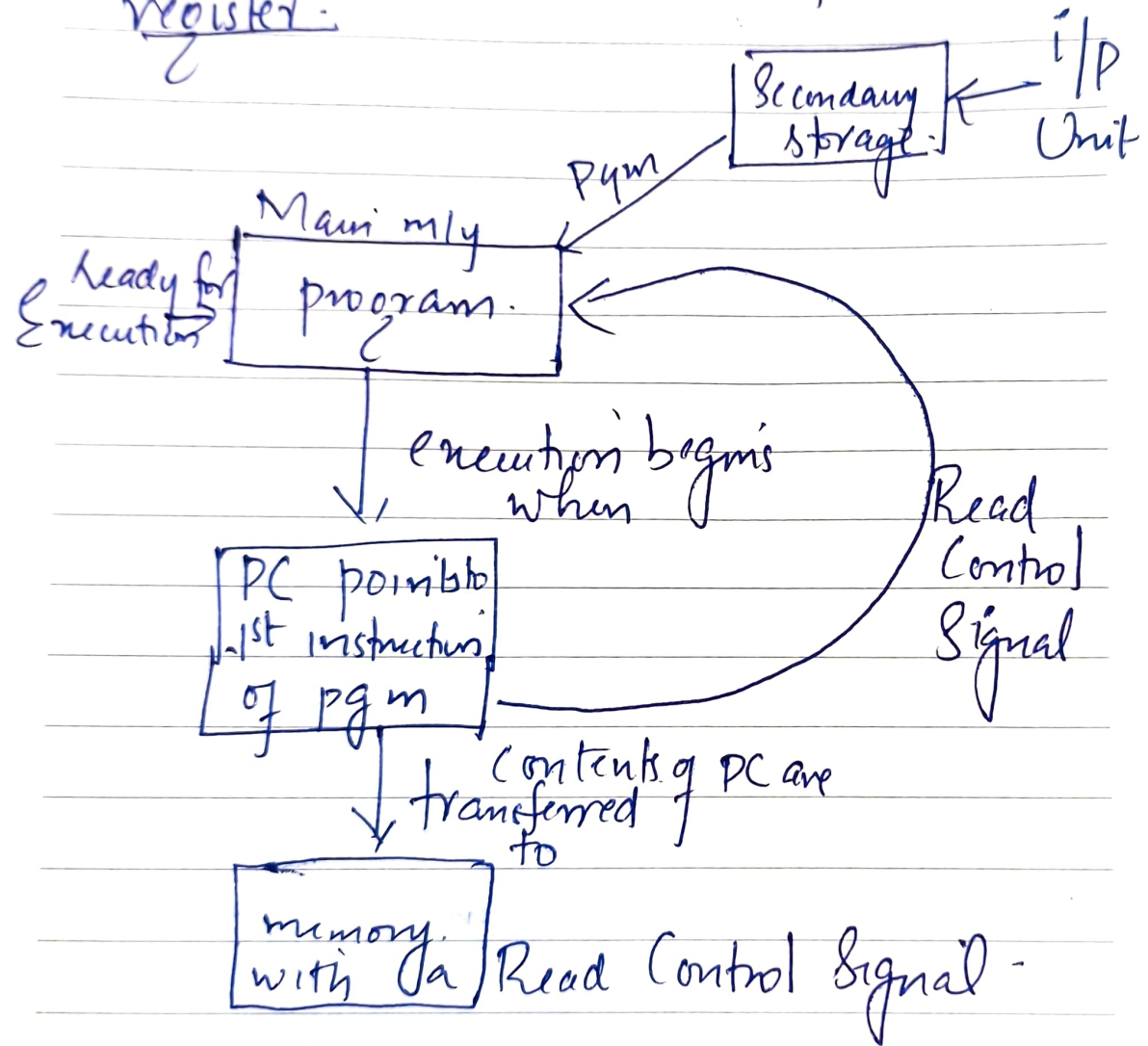
- $\rightarrow$ Called as processor registers
- $\rightarrow$ fast storage locations.
- $\rightarrow$ temporarily store data and addresses while CPU executes instructions.
- $\rightarrow$ (Holding operands that are loaded from the memory for processing)
- $\rightarrow$ (Arithmetic & Logic Operations)

4) Processor - memory Interface $\rightarrow$

- $\rightarrow$ Circuit which manages the transfer of data between the main memory and the processor.

$\rightarrow$ If a word is to be read from the m/y, the interface sends the address of that word to the memory along with a Read control signal.

↳ The i/f waits for the word to be retrieved, then transfers it to the appropriate processor register.

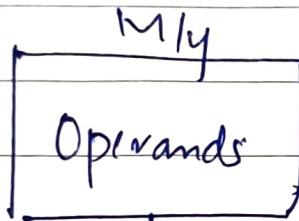


At this point, the instruction is ready to be interpreted & executed.

LOAD } Data transfer
STORE }

ADD → Arithmetic Operations

Operands



When operand is required for an instruction, it is fetched by sending its address to memory and initiating a Read Operation.

→ when the operand is fetched from the m/m, it is transferred to a processor register.

→ ALU perform the desired operations from the processor registers.

- ↳ Result is then transferred to another processor register.
- ↳ if the result is written into the memory with a Store instruction, it is transferred from processor register to the memory along with the address of the location where result is stored;
- ↳ Then a write operation is initiated
- ↳ While transferring data b/w memory and processor the computer accepts data from 'i/p' devices and sends data to 'o/p' devices.
- ↳ So, some instructions are provided for handling I/O transfers.

Normal execution of pgm
 ↑ to be preempted for urgent action
 * Interrupt → Eg:

↳ A monitoring device in a Computer-controlled industrial process may detect a dangerous situation;

↳ To respond immediately, execution of the current pgm must be suspended.

↳ To cause this the device raises an 'interrupt signal'.

↳ It is a request for service by the processor.

↳ The processor provides the requested service by executing a program called an

Interrupt - Service Routine.

↳ This may alter the internal state of the processor, its current internal state is saved in the memory before servicing the interrupt request.

↳ Information to be saved.

- ↓ Contents of the PC
- ↓ Contents of general purpose registers
- ↓ Control information.

↳ When the interrupt service routine is completed, the state of the processor is restored from the memory so that interrupted programs may continue.

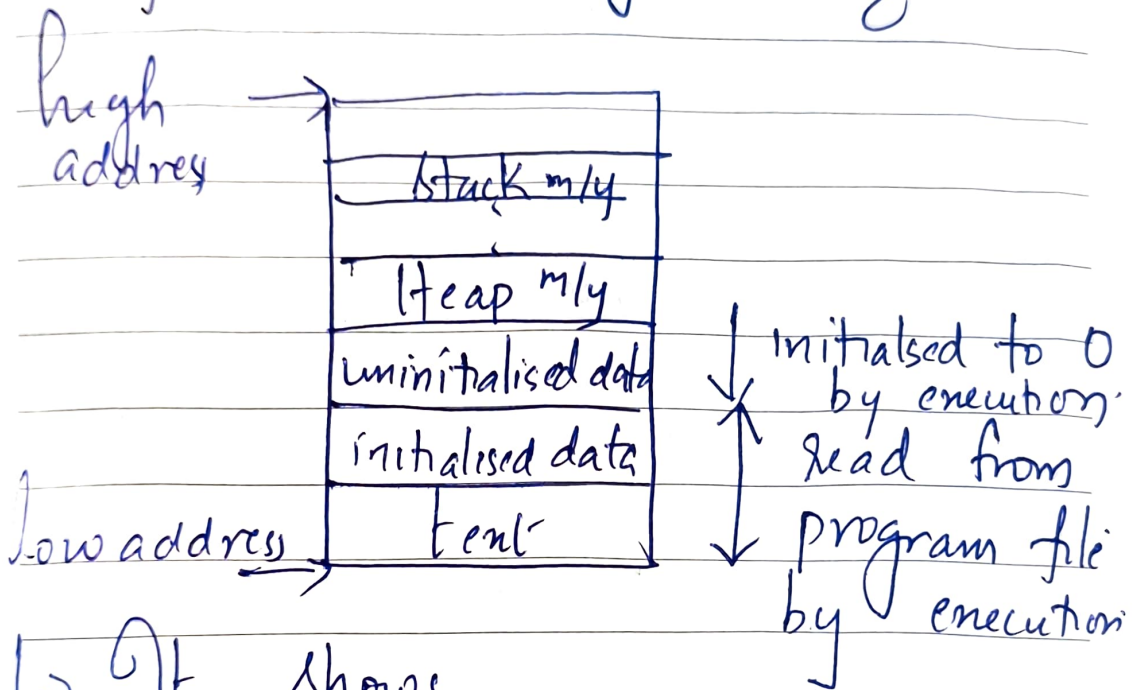
{ reg: MAR, MDR)

3. Memory Map

Date / /

SUN MON TUE WED THU FRI SAT

↳ Means a layout of mapping of how memory is organised.



↳ It shows,

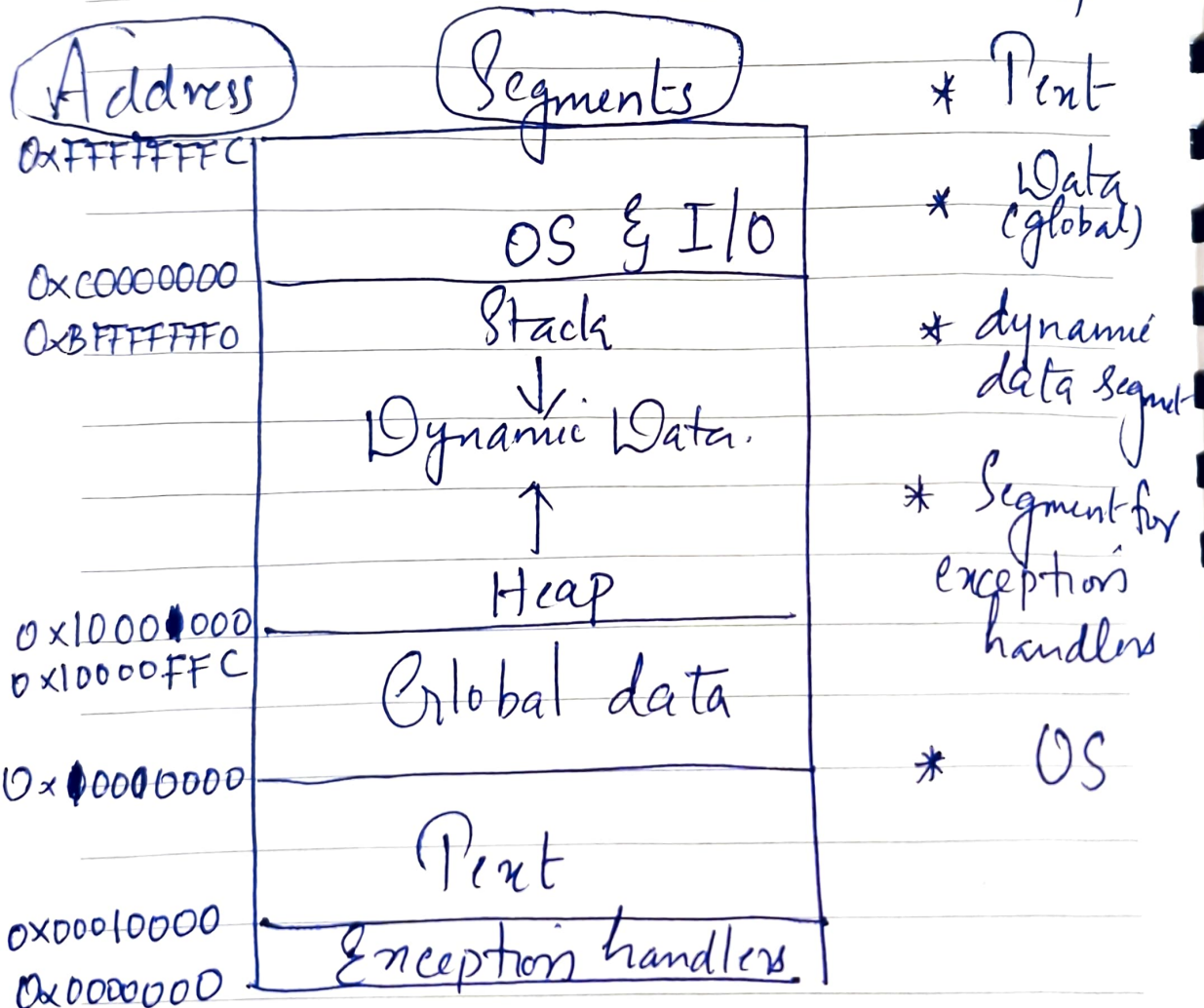
- Which part is used for Program Instructions
- Which part for data
- Which part for Stack/heap/OS
- RAM/ROM/I/O devices

↳ So it is the layout of memory is arranged in a Computer system.

↳ It assigns different address ranges for ROM, RAM, stack, heap

• It helps the CPU access data and devices correctly

• Memory map divides the address space into 5 parts or segments
↓



=> Can be used to identify,

- * Where the programs are loaded
- * Where the devices are located
- * Where the interrupts are controlled
- * How OS manages memory.

Memory areas.

Text Segment :- The executable code/instruction of pgm
(Read only Data)

Data :- Global & Static Variable,
Strings, Constants,
Initialised Data,
Uninitialised Data or ~~memory~~
Static Zero initialised Data

Heap :- Dynamically allocated memory
(malloc)
↑
Grows upward

Stack → Local Variables,
 Function call frames,
 Return address
 ↓
 Grows typically downwards

(highest part)

OS → RAM → Stores programs &
 & Data.
 &
 ROM → Stores boot loader

Memory
 mapped
 I/O

(Exception handlers
 run at start up.)

Memory Location, and Addresses

SUN MON TUE WED THU FRI SAT

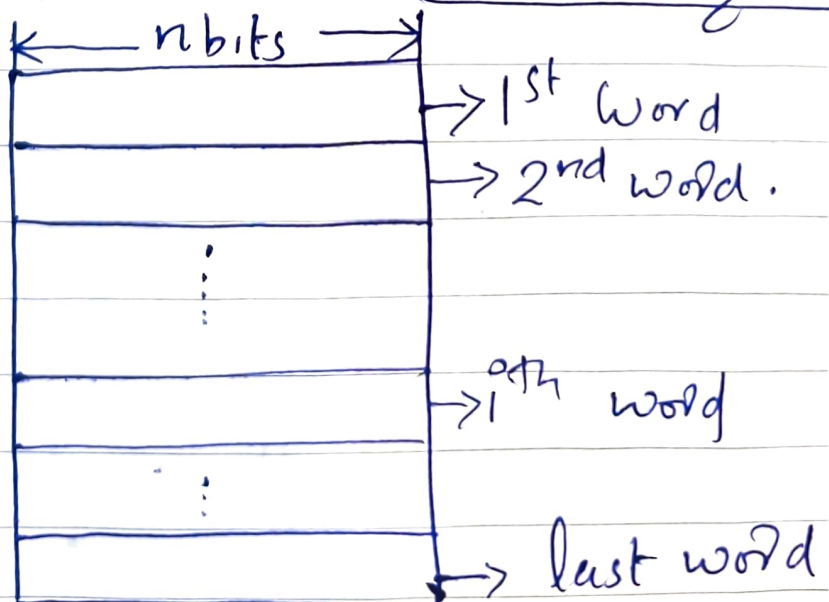
- * Memory = millions of storage cells, which can store a bit of information having value 0 or 1.

↳ How organised the memory?

- * groups of fixed size

- * group of 'n' bits can be retrieved in a single. This is called as a word of information.

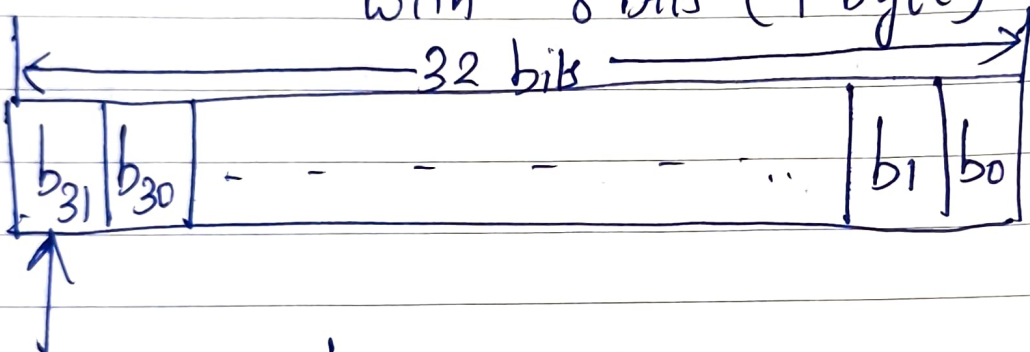
- * 'n' is called 'word length'



* Modern Computer's word length.
(16 to 64 bits range)

$\Rightarrow$ if wordlength = 32 bits

$\rightarrow$ either a 32-bit signed number
 $\rightarrow$ or 4 ASCII encoded characters
with 8 bits (1 byte)



Sign bit $\Rightarrow$ $b_{31} = 0$ for positive nos
 $b_{31} = 1$ for negative nos

fig: Signed Numbers.

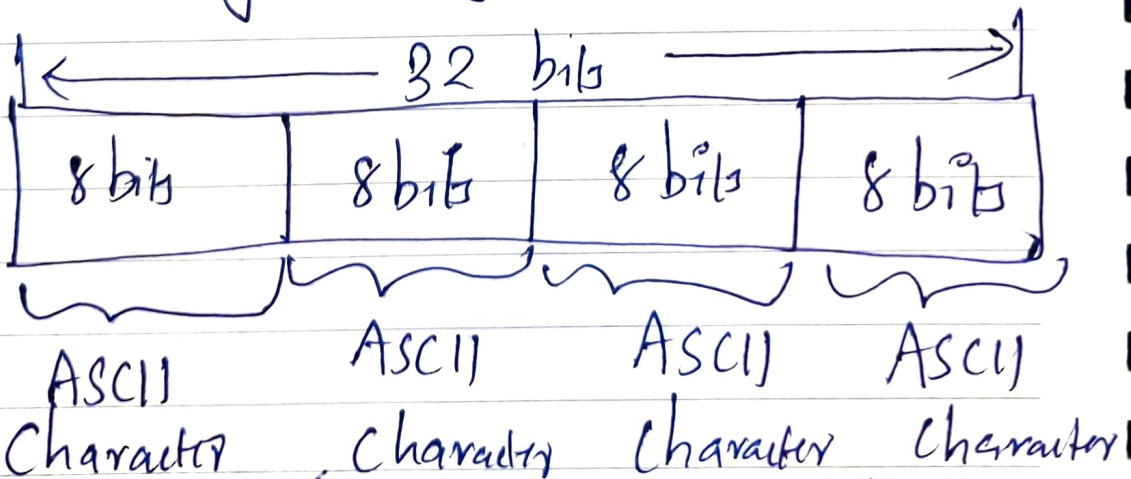


fig:

* Accessing the memory to store/retrieve a single item of information (either a word, or a byte) requires distinct addresses for each location.

for this we can

* Use 0 to $2^k - 1$ numbers, which means the memory can have upto 2^k addressable locations.

* 2^k is called the address space of the computer.

Eg: 24 bit address generates an address space of 2^{24} (16,777,216) locations.

$$= 16 \text{ M (16 Mega)}$$

$$[1 \text{ M} = 2^{20} = 1048576]$$

* 32 bit address creates an address space of 2^{32} or

$$2^{32} = 4 \text{ G (Giga)}$$

$$(1 \text{ G} = 2^{30})$$

* $2^{10} = 1024 = 1 \text{ kilo.}$

* $2^{40} = 1 \text{ Tera,}$

$$\Rightarrow 1 \text{ KB} = 2^{10} \text{ bytes (1024 bytes)}$$

$$1 \text{ MB} = 2^{20} \text{ bytes (1024} \times 1024)$$

$$1 \text{ GB} = 2^{30} \text{ bytes (1024} \times 1024 \times 1024)$$

$$1 \text{ TB} = 2^{40} \text{ bytes}$$

$$1024 \times 1024 \times 1024 \times 1024.$$

Byte addressability.

Date / /

SUN MON TUE WED THU FRI SAT

↳ 3 basic information quantities

- * bit
- * byte
- * word

→ A byte = 8 bits

→ Word length ranges 16 to 64 bits

⇒ why not give every bit an address?

↳ memory is huge, giving an address to every single bit would make things complicated

↳ So instead of assigning an address to each bit, computers assign an address to each byte (8 bits)

→ This is called byte-addressable memory.

Byte addressable means;

↳ Each m/y address refers to one byte.

↳ Byte address will be successive;
0, 1, 2, 3 - - -

↳ If the computer has a 32-bit word length, it means,

$$1 \text{ word} = 32 \text{ bits} = 4 \text{ bytes}$$

↳ Byte address will be 4 consecutive address

0, 4, 8, 12 - - -

[if 16 bit
means 0, 2, 4, 6]

Insight,

↳ m/y is divided into bytes, each byte has an address

↳ words (32, 64) are made by grouping multiple ^{bit} bytes.

Big Endian and Little Endian

SUN MON TUE WED THU FRI SAT

Assignment

Date

/

/

↳ Memory is byte-addressable
(each address refers to one byte)

↳ Word / Multibyte Value is stored in several consecutive bytes.

Endianness is the rule that decides which byte of the multibyte value goes into the lowest memory address.

↳ Big-Endian — (left most bytes) The most significant byte (MSB), contributes the largest value and is stored at the lowest address.

↳ Little Endian → The least significant bytes (LSB) contribute lowest address. (smallest value)

Eg: 0x12345678 (Hex Value)

MSB — 12 (left)

LSB — 78 (right)

Eg: 32 bit word = 4 bytes

	<u>0x12</u>	<u>34</u>	<u>56</u>	<u>78</u>	} Big Endian lowest address = MSB
Byte	0	1	2	3	
	↓	↓	↓	↓	
	0x12	0x34	0x56	0x78	
	(MSB)			(LSB)	

(LSB) 0x78 0x56 0x34 0x12 (MSB)

Byte 0 1 2 3

(little endian, lowest address = LSB)

Eg: MSB 15 36 58 79 LSB. (16 bits = 2 bytes)

0	15
1	36
2	58
3	79

Big endian

0	79
1	58
2	36
3	15

Little endian.

Examples

* If you store the 16 bit number 0x1234 at address 100

0x1234 (0x12 0x34)

	↓	↓
Big endian	100	101
Little endian	0x34	0x12
	↓	↓
	100	101